

ContextZip Compression Results

Real measurements on actual agent output -- bash, grep, web research, and live session compaction.
Data collected by two Haiku agents. Compact comparison runs on 172-message real session (30k tokens).

April 9, 2026 Profile: default 4 categories 172 messages benchmarked

COMPACT SAVING

92.2%

vs vanilla /compact -- 1,206
-> 94 ctx tokens

CONVERSATION

52.4%

Async/Python explainer -- 540
-> 257 tokens

RESEARCH TEXT

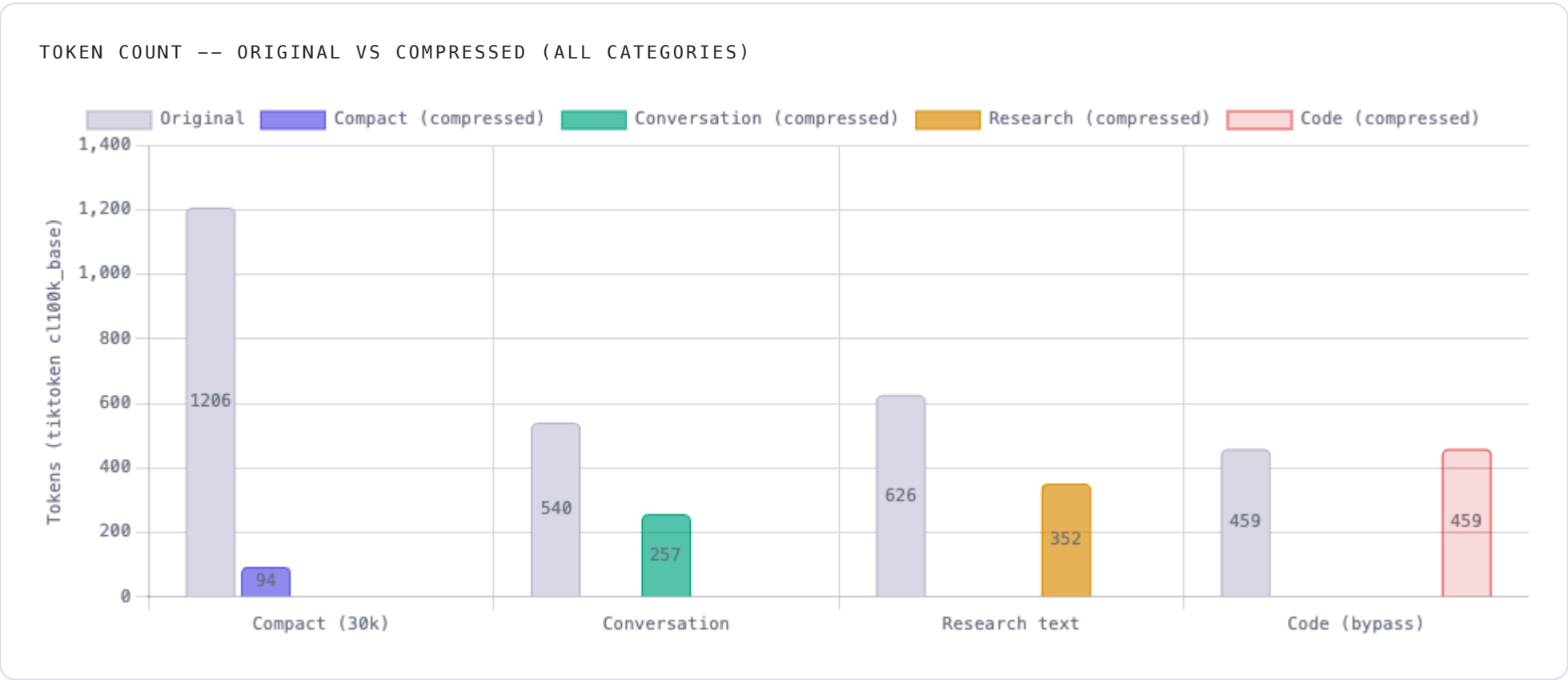
43.8%

Transformer paper -- 626 ->
352 tokens

CODE BYPASS

0%

Bash/git + grep -- preserved
verbatim



COMPACT COMPARISON -- VANILLA /COMPACT VS CONTEXTZIP HOOKS

After a 30k-token session (172 messages), both approaches run /compact. Vanilla Claude generates a full prose summary. ContextZip forces a 1-line summary and re-injects the last 10 archived turns -- compressed. What goes back into context:

Vanilla /compact
prose summary of full session

1,206 tok

back into context
~1,206 tokens

ContextZip compact
1-line + 10 archived turns

9 tok -- 1-line summary
85 tok -- last 10 turns (compressed)

back into context
94 tokens

Why bash output shows 0% reduction: ContextZip's code-bypass heuristic detected git diff output (high symbol density: paths, + markers, pipe characters) and returned it verbatim. Compressing file paths and diffs destroys their meaning. The bypass fires only on content that would be semantically damaged by deduplication.

METHODOLOGY		
DATA SOURCE	COMPRESSION PROFILE	TOKEN COUNTING

2 Claude Haiku agents -- live bash + web research + real session transcripts	default (stopwords: basic, pronouns, verbs, demonstratives, conversational)	tiktoken cl100k_base (OpenAI tokenizer -- exact counts)
VANILLA COMPACT ESTIMATE 4% of session tokens -- typical Claude prose summary for a technical session	CODE BYPASS RULE Triggers on: fenced code, symbol density >10 + indentation + newlines >2	PROTECTION AURA 3-word window around logic words (if, for, filter, return...)